Stalin: The Game

Robin Langer

1 September 2026

I am not an expert at Unix, or at web app development, or at container combinators, so these notes are just as much for myself as for anybody else.

Introduction

Neil Ghani's *Containers for Typed Agentic AI* describes an algebra: an interface is a pair — the prompts you may send, and, *for each prompt*, the type of reply that would answer it — and such interfaces compose, in four different ways. It is a construction in dependent type theory. TypeScript has no dependent types.

This note is about faking it anyway, and the claim it makes is that the fake is good enough to run a game on. Two ordinary things do the work: command-line tools, and `async` event handlers. Neither is exotic. Both have been in daily use for decades by people who have never heard of a container.

Here is the part worth reading for. When one level of the game asks the level below it a question, the question travels *downwards* as a real process: a program is started, a socket is opened, and the asking program then stops in the middle of a line and hands the machine back. When the answer arrives, it travels *backwards* up the same path, and each program resumes at the exact line it stopped on, with the reply in hand. Control down, data back up — which is precisely the shape the algebra says a morphism has, and it is one keyword, `await`, doing all of it.

That is the destination, and it is why the route is long. The fake is made of Unix and of HTTP, so those come first: what a process is, what a file descriptor is, what the shell actually does with a line you type, what an HTTP status code is for, and what a type system can and cannot promise about a value that arrived over a wire. None of it is difficult and none of it is assumed — every idea gets one subsection and one small example, and nothing is used before it has been introduced. Then the game, then the algebra, then the two put together.

The reward at the end is a five-server program in which *choosing which combinator to use is itself a game mechanic*, and in which the fake fails in exactly two places — visibly, and for reasons worth knowing.

1 The Unix philosophy, pipes, and CLI tools

In case the reader is a little rusty, we begin with a brief review of the Unix this note leans on: what a process is, how one reaches the world outside itself, how two of them are joined together, and why a command-line tool has the shape it has. None of this is specific to the game `stalin`, but all of it is used by the game. The game comes later, once the concepts are in place.

Each part below introduces one idea and, where it helps, one small example that illustrates only that idea. Nothing is used before it has been introduced.

A process

A *process* is one running program, together with everything the kernel keeps on its behalf: its memory, its position in the code, its identity — which user it runs as — and a number. A program is a file sitting on a disk, and does nothing; a process is that file in motion. Run the same program twice and you have two processes which share nothing and cannot see each other.

The number is the process's *pid*. It is how anything outside the process refers to it, and there is no other handle: to stop a program you send a signal to its pid.

User space and kernel space

A running machine is divided in two. Your code executes in *user space*, where it may compute freely and touch nothing: it cannot address a disk, a network card, or another process's memory, because the processor is in a mode that forbids it. The *kernel* executes in kernel space, where it may touch everything. The division is enforced by the hardware, not by convention.

There is exactly one interface between the two: the *system call*. It suspends your process, switches the processor into kernel mode, runs kernel code on your process's behalf, and returns. Every effect a program has on the world outside its own memory is a system call. There is no other mechanism.

`read` and `write`

That interface could have been wide. It is in fact very narrow. The two calls that matter most are these:

```
read (n, buffer, count)   -- give me up to count bytes from the thing numbered n
write(n, buffer, count)   -- take these count bytes and put them in the thing
    numbered n
```

Each takes a number identifying something already open, a place to put bytes or take them from, and how many. Between them they are very nearly the whole interface to the outside world. What that number is, and where it comes from, is the subject of two subsections below.

Everything is a file

Why should two calls be enough? Because of a design decision usually stated as a slogan: **everything is a file**. A file on a disk, a terminal, a network connection, a source of random bytes — all of them are presented to user space through the same interface, so the same two calls reach all of them.

The slogan overstates itself in one respect, and it is worth being precise about which. Things do differ in how they are *made*: a file is opened by name, a network connection has to be built by a short sequence of calls of its own. What they do not differ in is how they are *used*. Once the thing exists and you are holding its number, `read` and `write` are the whole vocabulary. The uniformity is at the point of use — which is the point that matters, because using is where one program meets another.

File descriptors

That number is a *file descriptor*. It is a small non-negative integer, and it is an index into a table the kernel keeps for each process, whose entries point at the things that process has open. Nothing in the number says what kind of thing it is. The descriptor is an opaque handle, and

that is exactly why one program can be pointed at a file one day and at a network connection the next without being rewritten.

Three entries are filled in before a process begins, and it did not fill them in itself — whoever started it did.

```
0   standard input    the stream the program reads
1   standard output   where its results go
2   standard error    where its complaints go
```

Start a program from a terminal and all three point at that terminal, which is why results and complaints appear jumbled together in one scrolling window. That is a property of the terminal, not of the program.

`cat`

Everything so far is enough to write a real Unix program. `cat` — named for *concatenate* — copies its input to its output, unchanged, and does nothing else. That is its entire specification, and it is almost exactly `read` and `write` in a loop:

```
char buf[4096];
for (;;) {
    n = read(0, buf, 4096);   -- take up to 4096 bytes from standard input
    if (n == 0) break;        -- 0 bytes means end of input; stop
    if (n <  0) exit(1);      -- negative means the read itself failed
    write(1, buf, n);         -- put exactly those n bytes on standard output
}
```

The real `cat` is longer, but the extra length is all flags and error messages; that loop is the program. Look at what it does *not* contain. It never asks what `0` is attached to, or what `1` is attached to, because `read` and `write` provide no way to ask. A disk file, a keyboard, another program's output: the loop is identical and the program cannot tell the difference.

`fork` and `exec`

Two more system calls, and they are the two that make new processes.

`fork` *duplicates* the calling process. Where there was one process there are now two, identical in their memory, their open descriptors and their position in the code, differing only in the value `fork` handed back — which is how each tells whether it is the original or the copy.

`exec` *replaces* the program running inside a process. Same process, same pid, same descriptor table, but the old code and memory are discarded and a new program is loaded in their place.

Neither is a "run this program" call, and that is the point: to start a program, a process `fork`s, and then the copy `exec`s. The descriptor table survives both, so the copy inherits the original's descriptors — and in the gap between the two calls the copy is still running the original's code, so it can rearrange its own table before replacing itself. Three subsections below turn on that gap.

The shell

The shell is not part of the kernel and has no privilege you do not have. It is an ordinary program running in user space with your identity, and its purpose is to start other programs.

That is all it is: a program whose job is running programs. Two things it relies on come first, and then what it does with a line you have typed.

The environment

One more thing a process carries: its *environment*, a set of named strings held alongside its memory and its descriptor table. They are called variables, but they are only ever text. Like the descriptor table, the environment survives `fork` and `exec`, so a new process starts with a copy of the one that started it:

```
$ export GREETING=hello     -- add it to this shell environment
$ sh -c 'echo $GREETING'    -- a different process, started by this one
hello
```

That is the whole of it: named text, inherited downwards. The most used of them is `PATH`, which holds a list of directories; the shell searches them whenever it is looking for a program to run.

Globbing

A word containing `*`, `?` or `[…]` is a *glob*: a pattern to be matched against the names of files that exist. `*` stands for any run of characters, `?` for a single one, `[abc]` for one character out of a set.

The matching is done by the shell, before any program is started:

```
$ echo *.txt
log.txt notes.txt other.txt
```

`echo` prints its arguments and nothing else, so what it printed is exactly what it was given: three filenames. It never saw an asterisk. That is why every program accepts patterns, and why no program contains any code for matching them.

What the shell does with a line

Given a line, the shell does five things, and every one of them has already been described.

1. **Split the line into words.** The first word names what to run; the rest become the argument vector that program will receive.
2. **Find the program.** The first word is looked for in each directory named by `PATH`, in order, and the first match wins.
3. **`fork`.** The shell duplicates itself.
4. **`exec`.** The copy replaces itself with the program. The shell that read the line is still there, waiting for it to finish.
5. **Keep the exit status.** When the program finishes the kernel gives the shell a small number it left behind, by convention zero for success.

One command cannot work that way, and the reason is worth seeing. `cd` has to be built into the shell rather than being a program, because a program runs in the copy, and the copy changing its own working directory would leave the shell exactly where it was. Anything that must alter the shell's own state has to be part of the shell.

A remark, because this is the mechanism behind something that catches people out with coding agents. Claude Code runs each shell command exactly as the shell runs a program: it forks, and the copy execs. A `cd` in that command changes the copy's working directory, and the copy then exits. So no command the agent runs can change the agent's. The directory it was launched in is therefore the directory it has for the whole session, which is why you must start it in the project you actually mean to work on.

Redirection

Now the gap between `fork` and `exec`. Writing `> log` after a command tells the shell: in the copy, and before exec'ing, open the file `log` and put it in slot 1.

```
$ echo hello > log      -- slot 1 is now the file, not the terminal
$ cat log
hello
```

`2> errs` redirects descriptor 2 in the same way. Nothing was added to `echo` to make this work, and `echo` cannot discover that it happened: it wrote to descriptor 1 exactly as it always does. Redirection is the shell's syntax and the shell's doing, from beginning to end.

Pipes

A *pipe* is a buffer the kernel will hand out on request, with two descriptors: one to write into, one to read out of. `|` is the same redirection as `>`, with a pipe where the file was. The shell asks for a pipe, forks twice, puts the writing end in the left program's slot 1 and the reading end in the right program's slot 0, and execs both.

```
$ cat notes.txt | wc -l
       3
```

`wc -l` counts the lines on its standard input. It was given no filename and does not know one exists.

Both programs run at the same time; `wc` does not wait for `cat` to finish. The buffer is finite — 64×1024 bytes on Linux — and the kernel makes each side wait for the other rather than growing it. When it is full, `cat`'s next `write` does not return until `wc` has consumed something; when it is empty, `wc`'s next `read` does not return until `cat` has produced something. Neither program contains a line of code about this.

Two things follow, and both are worth having in mind. A pipeline over a very large file needs no more memory than a pipeline over a small one, because at no moment does either process hold more than the buffer. And the second program starts answering immediately rather than at the end: in `cat huge.log | grep x`, the command `grep` is handed the first 64×1024 bytes as soon as `cat` has written them and reports its first match from those, while `cat` is still reading. It does not need to wait for `cat` to finish, and nothing in either program arranges for it not to. That is simply what two processes and one finite buffer do.

Exit status, and the shell's control flow

The small number a program leaves behind is what the shell builds its control flow on.

- `;` — run the next command regardless of the status.
- `&&` — run the next command only if the previous one exited zero.

- `||` — run the next command only if the previous one exited non-zero.
- `&` — start this command and do not wait for it, so several can run at once.

We could say a good deal more about all this, but it is not really relevant to the game `stalin`.

Shell scripts

A file of such lines is a shell script, and running it is the same as having typed them. So a pipeline that worked once can be kept and named. And because a script is started like any other program — found on `PATH`, `fork`ed, `exec`ed, its status collected — a script can appear in a pipeline itself, on either side, and nothing that uses it needs to know it is a script.

Three channels, not one

Gather up what a program actually offers the world outside it. There are three separate channels, and they carry three different kinds of thing.

Channel	Carries	Read by
descriptor 1 (stdout)	the results	the next program in the pipeline
descriptor 2 (stderr)	the narration	a person
the exit status	the verdict	the shell

Keeping them apart is what makes a program usable by another program: a pipe takes descriptor 1 and nothing else, so whatever reads from a program receives its results and never has to sort them out from error text. Putting results and narration both on descriptor 1 is the commonest way to make a program unusable in a pipeline.

Manual pages, and what is not checked

None of the above is checked by anything. The interface between two programs is bytes: any two can be joined, which is the strength, and nothing verifies that the composition means anything, which is the cost. What a program promises is written in its manual page and nowhere else, so the only thing enforcing it is that you read it.

Here is what that costs. You want to build only if the log has no errors in it, so you write this:

```
$ grep -c ERROR log && ./build
```

`&&` runs the next command only if the previous one exited zero, and `grep` exits zero when it *finds* something. So the line does the exact opposite of what you wanted.

```
$ grep -c ERROR dirty.log && ./build
1
  building                  -- errors present, grep exits 0, so it builds
$ grep -c ERROR clean.log && ./build
0                           -- no errors, grep exits 1, so it does not
```

It builds exactly when the log is dirty and refuses exactly when the log is clean. Both halves of `grep`'s behaviour are documented and neither is enforced, the shell did precisely what it was told, and nothing anywhere reports a problem.

What you wanted was `grep` answering *did you find anything* rather than *how many*. That is what `-q` is for: it prints nothing at all and reports only in its exit status — zero if it found at

least one match, one if it found none. Put the test where the shell can actually see it, and use `||` so the build runs when nothing was found:

```
$ grep -q ERROR log || ./build
```

Notice that `-c` was carrying that same exit status all along. The count it printed was never the thing `&&` was reading.

A note before leaving this. Claude is an absolute master at writing shell scripts, and this is genuinely a place where you do not need the details. Ask Claude for a script that builds only when the log is clean and you will get a correct one. What you do need is the concept.

None of this is enforced, and this is a deliberate design decision. The only thing holding any of it together is "developer discipline" — somebody read the manual page — and at this level of the stack that is the whole of the enforcement there is.

The dictum

Let's summarise what we have: one uniform interface reached by `read` and `write`, three separate channels, and a pipe that joins one program to another. With that on the table, the philosophy reads as a consequence of the machinery rather than as an aspiration. Doug McIlroy, who wrote the pipe into Unix v3 in 1973, put it in three sentences:

> Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.

Mike Gancarz later expanded this into nine tenets — small is beautiful; each program does one thing well; build a prototype as soon as you can; choose portability over efficiency; store data in flat text; use software leverage; use scripts to increase leverage and portability; avoid captive user interfaces; make every program a filter.

The last is the one that matters here, and it is the least obeyed. A *filter* is a program that reads a stream and writes a stream: its input is not a dialogue and its output is not a report to a human. `cat`, `wc` and `grep` are filters, which is why the examples above could be assembled without any of them having been designed for each other.

A CLI tool is already a dependent function

One last observation, and it is the one this note turns on.

A command-line tool receives one prompt — its argument vector — and produces a response whose *type depends on which prompt it was.* `git log` answers with a list of commits; `git status` answers with a working tree. Those are not the same kind of thing, and which kind you get is fixed by the word after `git`. That is $(p : P) \to R\,p$, with P the subcommands and R the result type of each. The interface (P, R) is what `--help` prints.

Notice what is *absent.* No event loop, no callbacks, no waiting, no concurrency. A CLI starts, is prompted once, answers, and exits. It is nonetheless event-driven in the sense that matters, because what makes a program event-driven is the prompt-indexed response type and not the waiting. Strip the loop away and this is what remains.

2 The Claude Code harness

Claude Code is a CLI process. When it runs a command it does what the shell does: it `fork`s, and the copy `exec`s, and the copy inherits its parent's identity.

That's it. It runs with the permissions of the user that executed it. It can read and write any file that user can read and write, and it can execute any command that user can execute.

In production one would create a user with limited permissions and run the Claude Code executable as that user.

3 HTTP and JSON

Again, in case the reader is a little rusty, we review some concepts from web app development: what a request and a response are, why so little of HTTP turned out to be the right amount, what a status code is for, which part of a request carries what, and what JSON does and does not preserve.

The examples below all run against a toy server that holds a list of books. It has nothing to do with the game; it exists so that each point can be made on its own.

A request and a response

HTTP is a request/response protocol. A client sends a request; a server sends a response; nothing else happens. Here is one complete exchange, the request and the response:

```
> GET /books/1 HTTP/1.1
  Host: 127.0.0.1:8099
  Accept: */*

< HTTP/1.1 200 OK
  Content-Type: application/json
  Content-Length: 53

  {"id": 1, "title": "Diaspora", "author": "Greg Egan"}
```

That is the whole protocol in one screen. A request is a *method* (`GET`), a *path* (`/books/1`), some *headers* (blue), and optionally a *body* (none in this example). A response is a three-digit *status* (`200`), some headers (blue), and a body (green).

A design that looked far too weak

It is worth a paragraph on where so little came from, because HTTP made the same kind of bet Unix made, and it looked just as inadequate at the time.

Tim Berners-Lee's first version, at CERN in 1991, had one method. You opened a connection, sent a single line — `GET /thepage` — and read the document that came back. There were no headers, no status codes, and no content types. The end of the response was signalled by the connection closing. That is the whole protocol. Methods, statuses and headers arrived in 1996, and persistent connections in 1997, but the shape did not change: one request, one response, and a server that remembers nothing about you afterwards.

Statelessness in particular looked like a straightforward defect. A protocol that cannot remember the previous request cannot have sessions, cannot have transactions, cannot notify you when something happens, and cannot check that what you sent was even the right kind of thing.

Statelessness is what lets the web scale. Because a request carries everything needed to answer it, any server can answer any request, so a hundred machines can sit behind one name. Because a response depends only on its request, anything in between can cache it. And because the server holds nothing on your behalf, a crash costs one request rather than your session. Load

balancing and caching were not added to HTTP later; they are consequences of the thing that looked like the flaw.

The best evidence is what has happened since. The encoding on the wire has been replaced twice — a binary framing in 2015, and a different transport underneath it in 2022 — and both times the semantics were left exactly as they were: a method, a path, some headers, a body, and a three-digit status. Two complete rewrites of how the bytes travel, and nothing above had to be rethought.

The parallel with the first section is close enough to be worth naming. A pipe carries bytes and does not know what they mean; an HTTP response carries bytes and a string claiming what they are. Both interfaces are universal precisely because they are weak, and in both cases whatever the interface does not supply has to be built above it by agreement between the two ends — *structure*, for the pipe, since nothing checks that two joined programs mean the same thing by their bytes, and *memory*, for HTTP, since every mechanism that makes a site remember you sits on top of the protocol rather than in it.

The method says what kind of thing you are doing

The method is the verb. There are several, and two of them are all this note needs.

Method	Means
`GET`	read something; change nothing
`POST`	create something

The distinction between them is the one that matters. A `GET` is meant to have no effect, which is why anything in the network is free to cache it or repeat it; a `POST` changes something, so nothing may do either. Asking the toy server for a book and asking it to add one differ in the verb, and the two answers differ accordingly:

```
$ curl -X GET  http://127.0.0.1:8099/books/1                    -- 200 OK
$ curl -X POST http://127.0.0.1:8099/books \
       -H 'Content-Type: application/json' \
       -d '{"title":"Permutation City","author":"Greg Egan"}'
{"id": 2, "title": "Permutation City", "author": "Greg Egan"}   -- 201 Created
```

The status code says what happened

The three-digit number is the server's answer to "how did that go". The first digit is the class, and the class is the part worth knowing.

Class	Means	Whose fault
`2xx`	it worked	—
`3xx`	look somewhere else	—
`4xx`	your request was wrong	the client's
`5xx`	my handling of it went wrong	the server's

The `4xx`/`5xx` split is the whole value of the vocabulary. It lets a caller tell "I asked for something impossible" from "the far end is broken", and those call for completely different responses: fix the request, or try again later.

Headers describe the message

Headers are name/value lines carrying facts about the message rather than the thing being asked for. Two appear in the exchange above.

`Content-Type` says what the body is: `application/json` here, `text/html` for a web page. It is a string, and it is a claim — nothing checks that a body labelled JSON is JSON. `Content-Length` says how many bytes the body has, so the reader knows where the message ends without waiting for the connection to close.

The useful discipline is the one the names suggest: a header is for what is true of the *message*, and the body is for the thing itself. Anything that is neither belongs in neither.

Where data can travel

There are four places a piece of information can ride in a request, and they are not interchangeable.

Place	Example
the path	`/books/1` — which thing
the query string	`/books?author=Egan` — how to filter or sort it
a header	`Content-Type: application/json` — about the message
the body	the JSON document itself

The one rule worth stating outright: never put a secret in a path or a query string. Both are part of the URL, and URLs are written to server logs, kept in browser history, and passed along in referrer headers. A password in a query string will end up in a log file that nobody thought of as sensitive.

Statelessness

Each request stands alone. The server does not inherently remember that you asked something a moment ago, and two identical requests get identical answers.

Everything that appears to contradict this is built on top. When a site remembers you are logged in, what happens is that the server sent a `Set-Cookie` header once and the browser now attaches `Cookie: session=abc123` to every subsequent request. The server looks that value up and says, in effect, "ah, user 42 again". Being logged in is not a state the protocol knows about; it is a string being re-sent, every single time, by a client that could just as easily send a different one. That last point is why a server must treat what arrives as a claim rather than a fact.

JSON

JSON — JavaScript Object Notation — is a text format for data. It has six types and no others: objects, arrays, strings, numbers, booleans, and `null`.

```
{                              // the whole record is an object
  "id":      1,                // a number
  "title":   "Diaspora",       // a string
  "author":  "Greg Egan",      // another string
  "inPrint": true,             // a boolean
  "tags":    ["sf"],           // an array
  "sequel":  null              // null
}
```

Being text is what makes it useful here: it is exactly what a pipe can carry and what a socket can send, so it crosses every boundary in this note unchanged.

What JSON does not carry

The omissions matter more than the six types, because each one is something that silently fails to survive the crossing.

No functions, and no "absent". Serialise an object with a function in it, or a field explicitly set to `undefined`, and they are simply not there afterwards. No error is raised:

```
const o = { id: 1, title: "Diaspora", author: "Greg Egan",
            reissue: undefined, describe: () => "a book" };
JSON.stringify(o)
// {"id":1,"title":"Diaspora","author":"Greg Egan"}
```

One number type. Every number is an IEEE 754 double. That is fine for a small id and not fine for money or for a large one:

```
JSON.stringify({ n: 0.1 + 0.2 })          // {"n":0.30000000000000004}
JSON.stringify({ big: 9007199254740993 }) // {"big":9007199254740992}
```

The second is not a rounding display: the value changed. Past 2^{53} the integers are no longer all representable.

No sum types, so a constructor becomes a convention. There is no notation for "this is one of the following kinds of thing". What people do instead is agree on a field:

```
JSON.parse('{"tag":"Book","title":"Diaspora"}').tag
// "Book"  -- a string, exactly like any other string
```

Nothing distinguishes that from a title that happens to read `"Book"`, and nothing requires the other fields a `Book` is supposed to have to be present.

No schema at all. There is no place in the format to say what the fields should be. Both ends may agree, but the agreement lives in prose and in code, exactly as in the first section.

Hence the shape of every crossing, in both directions. What arrives is a text document of unknown structure, and if anything is to be relied upon, something has to look. That is the subject of the next section.

4 TypeScript, event handlers, and servers

One more review, and this one is about what a type system can and cannot do for you at a boundary. The concepts are: what TypeScript checks and when, what a cast really is, how a value of unknown shape is turned into one of known shape, how a finite choice is written down, how a response type can depend on which request arrived, and what a server handler is. The examples continue with the shelf of books; none of them is from the game.

Types are checked before the program runs

TypeScript is a type system over JavaScript. The types are annotations: you write them, a checker reads them, and then they are removed and ordinary JavaScript runs.

```
interface Book { id: number; title: string; author: string }
const b: Book = { id: "1", title: "Diaspora", author: "Greg Egan" };
```

```
$ deno check t1.ts
TS2322 [ERROR]: Type 'string' is not assignable to type 'number'.
const b: Book = { id: "1", title: "Diaspora", author: "Greg Egan" };
                    ~~
```

Nothing ran. The mistake was found by reading the program, which is the whole point of having a checker, and it is the pleasant half of the story.

A cast is a promise, not a check

The types are *erased*. The program that executes has no representation of them and no check derived from them. So the `as` operator is not a conversion — it is an assertion that the checker accepts without verifying and the runtime cannot verify at all:

```
const raw = '{"id":"1","title":"Diaspora","author":"Greg Egan"}';
const b = JSON.parse(raw) as Book;   // a promise, not a check
console.log(b.id.toFixed(0));        // compiles: id is declared number
```

```
$ deno check t2.ts      -- passes. Nothing is wrong as far as the checker knows.
$ deno run   t2.ts
TypeError: b.id.toFixed is not a function
```

The checker was told `id` is a number and believed it. The id was a string all along, and the error surfaced at the first place that actually used it — which may be a long way from the cast, and in a real program is likely to be somewhere else entirely.

`unknown` is what you want at a boundary

Two types describe a value whose shape you do not know. `any` switches the checker off: every operation on an `any` is permitted. `unknown` does the opposite — it permits nothing until you have established what you have.

```
const raw: unknown = JSON.parse('{"id":"1","title":"Diaspora","author":"Greg Egan"
    }');
console.log(raw.id);
```

```
TS18046 [ERROR]: 'raw' is of type 'unknown'.
```

That refusal is the useful thing. `any` would have compiled and then failed later, in the manner of the previous subsection. `unknown` makes the boundary visible by refusing to let you cross it accidentally.

Validate, do not cast

So how does a value of unknown shape become one of known shape honestly? By code that looks at the fields. The signature to aim for is `unknown` in, and the type *or nothing* out:

```ts
function asBook(u: unknown): Book | null {
  if (typeof u !== "object" || u === null) return null;
  const o = u as Record<string, unknown>;
  if (typeof o.id     !== "number") return null;
  if (typeof o.title  !== "string") return null;
  if (typeof o.author !== "string") return null;
  return { id: o.id, title: o.title, author: o.author };  // rebuilt, not asserted
}
```

```
$ deno run t6.ts
accepted: Diaspora (1)
refused: {"id":"1","title":"Diaspora","author":"Greg Egan"}
refused: {"id":1,"author":"Greg Egan"}
```

Four things are deliberate. The parameter is `unknown`, so nothing can be done to it by accident. **Refusal is a value** — `null`, not an exception — so the caller has to decide what to do about it. The returned object is **constructed**, field by field, rather than asserted, so it is a `Book` because it was built as one. And the checks are what make the type true: after this function, a `Book` really does have a numeric id, and everything downstream may rely on it.

A tagged union writes down a finite choice

When a value is one of several kinds of thing, the kinds are written as a union, each carrying a field that says which it is:

```ts
type Query =
  | { kind: "count" }
  | { kind: "find";  title: string }
  | { kind: "byAuthor"; author: string };
```

The value of writing it this way is that the checker can tell the cases apart. Inside `if (q.kind === "find")` the type of `q` is the second member and `q.title` is available; in the other branches it is not, and asking for it does not compile.

The response type can depend on the request

Now the part that matters for everything above. Each of those three queries deserves a different kind of answer: a count is a number, a search is a book or nothing, a filter is a list. That is a function from the request to the *type* of the response, and over a finite set of cases it can simply be written down:

```ts
interface Answers { count: number; find: Book | null; byAuthor: Book[] }
type Reply<K extends Query["kind"]> = Answers[K];
```

`Reply<"count">` is `number`, `Reply<"byAuthor">` is `Book[]`. Read `Answers` as the table of a function from the tag to a type.

The handler table, total by construction

A program answering those queries is a function whose result type depends on its argument. Over a finite tag set it can be written as a record, one entry per case:

```
type Handlers = { [K in Query["kind"]]: (q: Extract<Query, { kind: K }>) => Reply<K
    > };

const handlers: Handlers = {
  count:    (_q) => SHELF.length,
  find:     (q)  => SHELF.find(b => b.title === q.title) ?? null,
  byAuthor: (q)  => SHELF.filter(b => b.author === q.author),
};
```

Each entry is checked against its own case: inside `find`, `q` is the `find` member and the return type is `Book | null`, not a union over all three.

And the table is total, not by anyone's diligence but by the shape of the type. Add a fourth query and the object stops satisfying `Handlers`:

```
TS2741 [ERROR]: Property 'oldest' is missing in type
       '{ count: ...; find: ...; byAuthor: ...; }' but required in type 'Handlers'.
```

There is no default branch to fall through and no dispatch to forget. A whole class of mistake becomes unwritable rather than merely untested.

A server handler, and who owns the loop

A server is the same idea with the request arriving over a socket. What you write is a function from a request to a response; the runtime supplies everything else — the socket, the accept loop, the parsing, the concurrency:

```
Deno.serve({ port: 8098 }, (req: Request): Response => {
  if (req.method !== "GET") return json({ error: "GET only" }, 405);
  return json(SHELF, 200);
});
```

```
$ curl -i http://127.0.0.1:8098/
HTTP/1.1 200 OK
content-type: application/json

[{"id":1,"title":"Diaspora","author":"Greg Egan"}]

$ curl -X POST http://127.0.0.1:8098/     -- 405
```

That is what "event handler" means operationally: inversion of control, with the loop on the far side of the library boundary. But notice that the loop is not what makes it event-driven in the sense that matters. A command-line tool has no loop at all and is event-driven in the same sense. What both have is a response whose type is indexed by the prompt. The loop is scheduling; the indexing is the type.

Where the type system stops

Put the two halves of this section together and the shape of the whole problem appears.

On the inside, the checker is strong. It computes which answers belong to which requests, makes a handler table total, and turns a class of mistake into a compile error.

On the outside, at the wire, it contributes nothing whatsoever. Everything arriving is `unknown`; the checker ran before the process started and has no representative in it. A single unchecked cast at that boundary forfeits every guarantee inside it, because from then on the checker is reasoning about a claim rather than a fact.

Which is why the whole of it rests on the smallest piece: a function with the signature `unknown → T | null`, that looks at the fields, and refuses.

Summary of the concepts

One shape recurs at every level of this stack: **a prompt determines the type of its response**. What changes from level to level is only who enforces it.

Level	Prompt	Response	Enforced by
a system call	`read`/`write` on a descriptor	bytes, or an error	the kernel
a pipe	the bytes upstream	the bytes downstream	nobody
a CLI tool	the argument vector	stdout and an exit status	a manual page
an HTTP request	method, path, headers, body	a status and a body	the status code
a handler	a tagged request	the answer at that tag	the compiler

The enforcement gets stronger as you read down that table, and it also gets narrower. The kernel checks everything and understands nothing: it will not let you read a file you have no right to, and it has no opinion whatever about whether the bytes mean anything. The compiler understands everything and checks only what never left the process: it knows exactly which answer belongs to which request, and it has gone home before the first byte arrives.

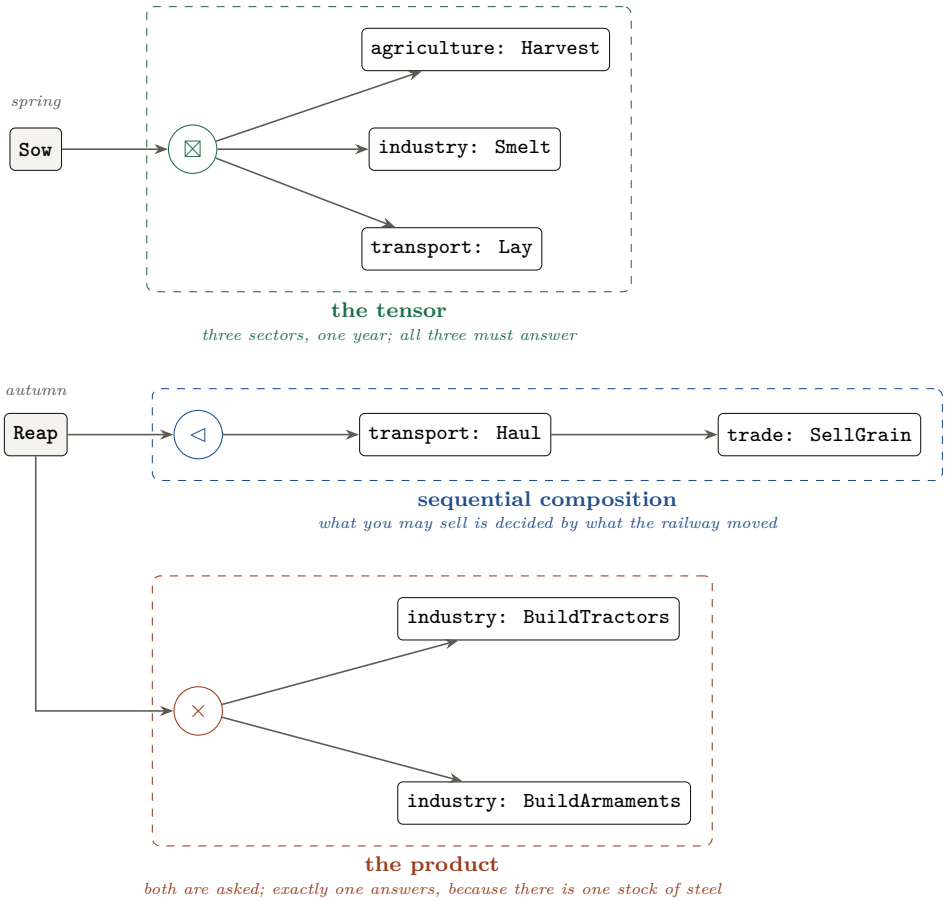
Neither end can be dispensed with, and neither reaches the other. So the engineering is all at the handovers, and there are three of them in this note.

- **From a pipe to a program.** The interface is bytes, so any two programs compose and nothing checks that the composition means anything. What makes a program usable by another program is a discipline nothing enforces: results on descriptor 1, narration on descriptor 2, a verdict in the exit status, and a manual page somebody has read.
- **From a program to a network.** HTTP adds exactly one thing the pipe lacked: a three-digit number saying what happened, and in particular whether the fault was the caller's or the server's. It is a small addition and it is the difference between a caller that can respond sensibly to failure and one that cannot.
- **From the network to the type system,** and this is the important one. What arrives is text of unknown structure. A function of type `unknown → T | null` that looks at the fields, rebuilds the value rather than asserting it, and returns `null` when the value is not what was claimed, is the only thing that makes the guarantees on the inside mean anything at run time. Cast instead, and the compiler spends the rest of the program reasoning carefully about something that was never true.

5 The game

The purpose of this game is to illustrate Ghani's theory of container combinators for event-driven programming. The game is the example; the algebra is the subject.

This is a game about Stalin and his First Five-Year Plan. You are given the Soviet Union in 1928 and five years to industrialise it, and you must also win the war in 1941. This was a tragic episode of history and I don't wish to make light of the story. It is for illustrative purposes only.



You want to build tractors. Not because tractors win anything, but because they raise the harvest *without* taking the grain: they are the way to pay for industrialisation in machines instead of in people. Everything else in the economy exists to get you to them.

For that you need steel and railways. Steel needs a mill, and a mill needs a German engineer, and an engineer has to be paid in hard currency. Hard currency comes from selling grain to the USA — the same grain the villages eat. Railways need steel too, and they are not capital but *throughput*: grain the railway cannot move rots at the sidings, whatever the harvest was, so track comes before tonnage. Early on you may also buy steel abroad outright, paying in grain, which is the same trade at a worse rate.

You must also feed the peasants. The labour force is one pool of people. They can work the fields, drive the tractors, work the mill or lay the track — but everyone eats and only the first two grow anything, which is why moving a peasant into the mill costs you twice. How hard the villages are squeezed is a decision you make every spring, and what is not taken is what they eat. Starvation in this game is not a scoring gimmick; it is the direct arithmetic consequence of the extraction decision, which is what it was.

And you must prepare for the war. In 1941 the country is invaded, and what it has put by decides the cost. Peasants serve as the soldiers, so the same population is both the workforce and the army, and steel not spent on tractors or track can be spent on armaments instead. Fight with too little steel and you win by sending more men; fight with enough and you send fewer. Fail to arm at all and no amount of mechanisation saves you.

So the aim is two numbers at once: the harvest at the end of the plan should exceed the harvest before it began, and as few people as possible should die getting there — counted in both columns, the starved and the fallen. **There are various strategies**, they disagree with each other, and the rest of this section is what happens when you try them.

A year is two turns, and the split is the point. In *spring* you fix where the hands go and how hard the villages are squeezed — *before* you know the weather. In *autumn* the harvest comes in graded, and you dispose of it knowing exactly what it was. A quota set in March against a harvest that fails in August is not a game mechanic; it is how a famine is administered. Five years is ten decisions, and then a reckoning.

Why the two costs are not independent

Combat power in 1941 is

$$m \cdot \text{manPower} \;+\; \text{steelPower} \cdot \sqrt{W \cdot m}$$

for m men and W tonnes of war reserve. A man is worth something holding nothing; a shell is worth nothing with nobody to fire it; and because the second term grows as a square root, the more steel each man already carries the less the next tonne adds. Men and steel are therefore complements, and partly substitutes: the same victory can be bought with more of one or more of the other, and the price of the second is paid in lives.

The trap is that extraction feeds both sides of that expression with opposite signs. A peasant starved in 1932 is a soldier missing in 1941, so squeezing harder to build steel shrinks the army the steel was meant to protect. Push far enough and you lose the war holding the largest stockpile in Europe.

Build them or buy them

There are two roads to a tractor, and choosing between them is most of the early game.

Build them. Hard currency buys a German engineer, the engineer raises the mill, the mill makes steel, twenty spare tonnes of steel opens the works, and the works makes twenty tractors a year forever. Four steps, and the first tractor is years away.

Buy them. Ten in hard currency each, ready to drive, needing no engineer and no mill and no works — and building none of those either. Every one is bought again next time.

Seven plans

A plan keeps its letter for the whole game, so a sequence of ten letters is a strategy and the space can be searched directly. All seven below were run at seed 1928, and the numbers are what came out of the terminal.

strategy	letters	tractors	armaments	starved	fallen	**dead**	outcome
gentle armament	`AJBEBJAJAJ`	2	89	5	4	**9**	won
brutalist mech.	`CMKEBIBJNJ`	22	80	14	5	**19**	won, mechanised
no railway	`KABEBIBJAJ`	22	82	25	4	**29**	won, mechanised
partial armament	`CABEBINJAA`	42	34	21	14	**35**	won, mechanised
totally brutal	`KJBEAJBJAJ`	2	78	44	5	**49**	won
bought tractors	`AMAMAFAMAF`	18	0	3	57	**60**	*lost*
never armed	`CAKEBINIAI`	62	0	15	52	**67**	*lost*

Five of the seven win, and the table says several things at once.

Never arming loses. A beautifully mechanised country — sixty-two tractors and the largest harvest on the board — falls in 1941, because men with nothing to fire cannot reach the enemy's strength.

Buying the tractors abroad is the humane road, and it is fatal. Three starve, the fewest of any line. But imported tractors build no mill, no engineers and no steel, so there is nothing to arm with, and fifty-seven soldiers pay for the three peasants.

Partial armament wins and bleeds. Thirty-four tonnes is enough to win and not enough to win cheaply: fourteen fallen against the best line's four. The war curve is smooth, so half an army is a victory paid for in men.

The railway comes before the steel. Grain the railway cannot move rots at the sidings. A hard quota with no track to carry it starves twenty-five people for nothing — worse than the same brutality with a railway, for the same industrial result.

The two rankings disagree. By soldiers lost, *no railway* and *gentle armament* tie at four; by everyone lost, gentle armament wins outright at nine while no railway comes third at twenty-nine. The game keeps both numbers and does not say which is the real one.

The plan the game was built around, and the one that beats it

The design intent was that *brutalist mechanisation* should be optimal: starve the villages early, spend it on rail, engineers and tractors, then turn the surplus into an army. It is morally uncomfortable and it was supposed to win.

It does not. `AJBEBJAJAJ` — found by machine search over the menu rather than by hand — wins for nine dead by doing almost nothing: put hands in the mill, buy the Germans once, then smelt straight into armaments for four years. No railway, no tractors on purpose, and no famine beyond the baseline five who die whatever you do.

But look at what it does not end up holding. A plan counts as mechanised if its final harvest beats 1928's, and gentle armament finishes at 258 against a baseline of 277: it feeds fewer people in 1932 than the country fed before the plan began. It wins the war and it does not build the country. Brutalist mechanisation ends at 285, with twenty-two tractors and an economy that would go on producing, and it pays five more lives for that. So the cheapest plan in lives is also the one that leaves nothing behind, and the game does not resolve that either.

The reason gentle armament wins at all is a dependency length rather than a number. Its chain is mill → steel → armaments: three links, so the first armaments are built in 1929. The mechanised chain is rail → export → engineers → works → tractors → freed hands → mill → steel → armaments: nine links, each costing at least a turn, against ten turns in the whole plan. The mechanised economy matures in 1932, and then the plan ends. This is recorded as a failing test in `calibrate.ts` rather than quietly redefined.

6 Containers and composition

Everything from here on is one idea from Neil Ghani's *Containers for Typed Agentic AI*, and the reader has met it already without its name.

A container is a pair

A *container* is a pair

$$P : \mathrm{Type} \qquad\qquad R : P \to \mathrm{Type}$$

of *prompts* P and, for each prompt, the type $R\,p$ of replies appropriate to it. An inhabitant of the interface is a dependent function

$$(p : P) \to R\,p.$$

That is the shape of the fourth section of this note, arriving with a name. The `Query` union was P. `Answers` was R. The handler table was $(p : P) \to R\,p$, written as a record because the index was finite. Ghani's observation is that event-driven programming "has a feature that turns out to be exactly a dependent type": which responses are appropriate depends on which event arrived. Nothing about loops or callbacks appears in that sentence, which is why a command-line tool is an instance of it too.

And a game is a container. P is the situations you can be in, $R\,s$ is the moves legal at s, and a player is $(s : P) \to R\,s$. That is not an analogy; it is the same type.

A morphism is how one container calls another

A morphism $(P, R) \to (Q, T)$ is a pair of functions:

$$u : P \to Q \qquad\qquad f : (p : P) \to T\,(u\,p) \to R\,p$$

u *delegates* — it turns my prompt into one for the level below. f *amalgamates* — it turns their reply into mine. Control flows *down* through u, and data flows *up* through f.

That shape is worth dwelling on, because it is the shape of a command economy, which is why the fit with the subject is good rather than decorative. The Plan does not do the harvesting. It converts "feed the towns" into "agriculture: reap", and converts the reply back into a number for its own ledger.

Four ways to compose

Containers compose, and four ways matter. Each is defined by what it does to the shapes and, separately, by what it does to the positions — and the second column is where the interesting differences live.

	shapes	**positions**
$+$ sum	disjoint union: one side or the other	each keeps its own
$\boxtimes$ tensor	multiply: a prompt for each side	multiply: a reply from *each*
$\lhd$ sequential	a first prompt, and a function choosing the second	a pair of replies
$\times$ product	multiply: prompt both	*sum*: exactly *one* answers

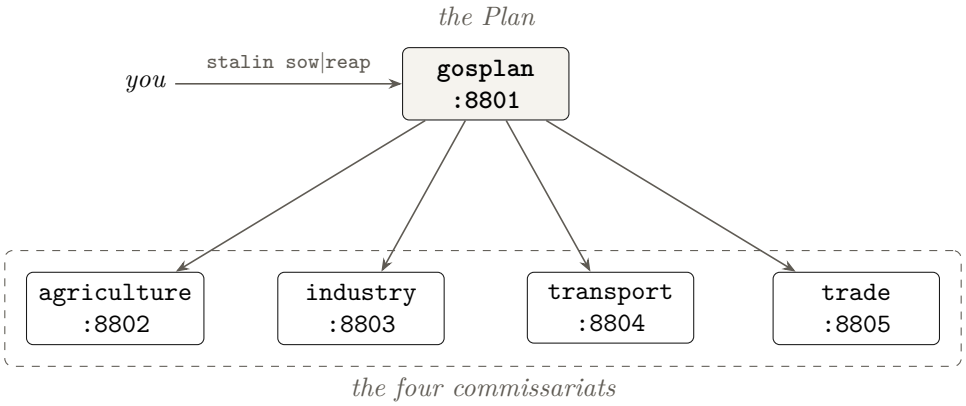
The distinction that does the most work is $\boxtimes$ against $\times$. Both prompt everybody, and their shapes are identical. The tensor requires all of them to answer; the product requires exactly one. Same question, different obligation — and in a resource allocation problem that is exactly the difference between doing several things at once and choosing between them.

$\lhd$ is the subtle one. Its shape is not a pair of prompts but a first prompt *together with a function* computing the second from the first's reply. So the second question is not knowable when the first is asked, which is what makes it sequential rather than parallel — and, as the next section shows, what makes it the one combinator that cannot be sent over a wire.

7 How the game is implemented

Five servers, and why they are servers

The game runs as five programs, each listening on a port. The player talks only to the Plan; the Plan delegates to the four commissariats beneath it.



Every arrow is a delegation, and every arrow points away from the Plan: the Plan is the only client any of them has. The four do not talk to each other, and none of them can see the others' books — which is what makes the picture a star rather than a chain, and what makes the levels separable in the first place.

They could have been five modules with a function call at every hop. The comment at the top of `wire.ts` says why they are not:

> A hop costs a process, which is the whole point: control genuinely flows down, and the boundary is real rather than a function call wearing a costume.

The delegate leg u maps *shapes*, and shapes are exactly what survives a wire. If a hop were a function call, nothing would stop a handler reaching into its caller's state, and the claim that the levels are separate would be untestable. Making it a socket makes the claim falsifiable: whatever crosses must be expressible in JSON, and the compiler on the far side has never heard of your types. The cost is written down too — about a fifth of a second per hop, which a hundred-game search pays forty times per game, hence an environment variable that keeps the HTTP hop and skips only the process spawn. No game a player sees uses it.

The game as a composite

Each turn is one of the four compositions, and which one it is makes a claim about the year rather than being a detail of the code.

	composite	**why that one**
spring	(agri $\boxtimes$ mill) $\boxtimes$ rail	all three work at once and *all three owe a report*; there is no partial year
autumn	transport $\triangleleft$ trade	what may be sold is computed *from* what the railway actually moved
build	tractors $\times$ armaments	the same steel, one decision: offer both, answer one

The product is the one to look at twice. Tractors and armaments are both offered every autumn and exactly one is built, because there is one stock of steel. Had the game let you split the steel between them it would have been a tensor, and the central dilemma of the previous section would have dissolved into a slider. The choice of combinator *is* the game mechanic.

The sum, honestly, is never built as a composite: there is no `sumC` call in the game. It appears instead as each container's own shape type, a tagged union, whose eliminator is the `switch` in every server. That is a coproduct doing real work, but calling it a use of the combinator would be an overstatement.

What crosses, and what the status code says

Every hop is a `POST` to a port with a JSON body, and the reply is JSON. The path is unused: the port identifies the container. The three outcomes are genuinely different kinds of thing, and the status vocabulary already distinguishes them.

```
try { body = await req.json(); }
catch { return json({ error: "not JSON" }, 400); }

const prompt = opts.parse(body);
if (prompt === null) {
  // The validator refused.  No cast was made, so nothing ill-typed is
  // downstream of this line -- that is the whole point of validating.
  return json({ error: "not a valid prompt for this container", got: body }, 400);
}
try   { return json(await opts.answer(prompt, depth + 1), 200); }
catch (e) { return json({ error: msg }, 500); }
```

Against the transport commissariat, with the servers running:

```
$ curl -s -X POST localhost:8804 \
    -d '{"tag":"Haul","railCapacity":40,"available":100,
         "needIndustry":30,"offerPort":50,"year":1929}'
{"toIndustry":30,"toPort":10,"stranded":40,"short":0}          -- 200

$ curl -s -X POST localhost:8804 -d '{"tag":"Haul","railCapacity":"lots"}'
{"error":"not a valid prompt for this container", ...}         -- 400
$ curl -s -X POST localhost:8804 -d 'not json at all'
{"error":"not JSON"}                                           -- 400
$ curl -s -X POST localhost:8804 -d '{"tag":"Purge"}'          -- 400
```

Read the first reply, because it is the game in four numbers. Forty units of rail capacity, a hundred units of grain in hand, thirty needed at the mill and fifty offered to the port: the mill is fed first, which leaves ten units of capacity, so ten reach the port and **forty tonnes are stranded** — grain the state owns, has procured, and cannot move. That is why a railway is throughput rather than capital, and it is the whole content of the *no railway* line in the table two sections back.

The three 400s are all "that was not a prompt", at three depths: not JSON at all; JSON but not this container's shape; a tag this container does not have. Only a 500 means "that was a prompt, and answering it went wrong". Getting that split right is what lets a caller tell a bug in itself from a bug in its callee.

One thing rides outside the body. A depth counter travels in a header, `x-tower-depth`, and the reason given in the source is worth adopting generally: *the depth rides in a header rather than in the prompt, because the prompt is the container's shape, and nothing that isn't a prompt belongs in it.*

A hop is a CLI tool

The program that carries a prompt to the next level is forty lines long and knows nothing about the game:

```
$ deno run --quiet --allow-net --allow-env \
    lib/delegate.ts 8804 '{"tag":"Census"}' 0
{"workforce":{"fields":0,"drivers":0,"mill":0,"rail":0,"dead":0},
 "stocks":{"railCapacity":0},"trueOutput":0}
```

It is the delegate leg u made executable, and it observes every discipline from the first section. Its exit codes distinguish the two kinds of failure: `2` for a malformed prompt or missing arguments, `1` for a port with nothing behind it. Every diagnostic goes to descriptor 2, and descriptor 1 carries the reply and nothing else — which is exactly what lets the caller parse the whole of stdout as one JSON document.

That discipline is also what makes the tracing possible. The servers narrate themselves on descriptor 2 while descriptor 1 stays machine-readable:

```
↓ gosplan   Sow order={"procurement":"firm","labour…":{"
  ↓ gosplan   ((agri (x) smelter) (x) transport)
    ↓ agri      Harvest workforce={"fields":100,...} tractors=0 year=1928
    ↑ agri      {grain, grade, perWorker, withoutTractors}
    ↓ industry  Smelt workers=12 millCapacity=8 year=1928
    ↑ industry  {steel, idleCapacity}
↑ gosplan   {kind, year, harvest, steel, …}
```

Indentation is the depth counter, ↓ is control flowing down and ↑ is data flowing back up. That is a derivation tree of container morphisms, printed by a program that had no idea it was doing that.

Where the compiler carries a rule of the game

Two rules are enforced by the type checker rather than validated at run time, and both work by the trick from the fourth section: a conditional type over a finite union.

```
export type ExportChoice<G extends HarvestGrade> =
  G extends "failure"  ? "none"
: G extends "poor"     ? "none" | "surplus"
: G extends "adequate" ? "none" | "surplus" | "full"
:                        "none" | "surplus" | "full" | "maximum";

exportPlan("bumper",  "maximum")   // fine
exportPlan("failure", "maximum")   // does not compile
```

Exporting grain during a failed harvest is not a move the engine rejects. It is a move that cannot be written down. The same trick carries the arc of the whole game: you may buy steel while in deficit, and never sell it, because a country that exports its steel is not industrialising.

```
export type SteelTrade<P extends SteelPosition> =
  P extends "deficit" ? "none" | "buy" : "none";
```

Both work only because the index set is finite. A rule over tonnages could not be typed this way; the same rule over *grades* can. That is also how the apparatus actually worked: quotas were set against reported categories, never against measured tonnes.

The one thing that cannot cross

Of everything JSON leaves out, one omission bites structurally rather than merely requiring care. A function cannot be serialised — and the shape of $\triangleleft$ is a first prompt together with a function computing the second.

So sequential composition cannot be distributed over the wire. It is resolved *above* it, in the runner, which prompts the first container and applies that function to the reply to obtain the second prompt; only the two ground prompts ever cross. The wire format is not neutral about which combinators can be sent and which must be interpreted locally, and this is the sharpest example in the repository of a mathematical structure meeting a transport and losing an argument.

What the reports are worth

Each commissariat holds two things the Plan cannot see: how well it is actually doing, and how much it inflates its dispatches. The Plan keeps the true ledger for grain, because the grain physically exists. What diverges is the *report*, and you plan against the report.

```
Narkomtiazhprom: quota fulfilled, with difficulties overcome.
```

This is the honest limit of everything above. The type of a dispatch guarantees its shape and says exactly nothing about its truth. A validator can establish that a reply is a well-formed dispatch; no validator can establish that the number in it is the number that happened. The reckoning prints both columns and leaves the reader to notice.

And where the protocol is not doing the work

One last piece of honesty. HTTP is stateless and these servers are not: each keeps its books at module scope, in the process. That is defensible — a commissariat *is* a stateful thing, and the exercise is about the container algebra rather than about session management — but it has a consequence, and the consequence is visible in the interface. The protocol cannot clear that state, so clearing it has to become a prompt:

```
Reset: (p) => { seed = p.seed; books = new Books(...); laid = 0; lastQuota = 0;
                return { ok: true }; },
```

`Reset` has no meaning in the fiction; no plan resets a railway. It exists because starting a new game must reach the commissariats, or the weather and the books carry over into the next one — a bug the comment records as having actually happened. Determinism on the seed is what makes a playthrough a regression test, so this is load-bearing rather than tidying. And the general lesson is the one worth taking away: when process state substitutes for protocol state, the lifecycle leaks into the interface.

Conclusion

What has been built is a *fake*. Ghani's container combinators are a construction in dependent type theory, and TypeScript has no dependent types. Nothing here added them. What the game does instead is impersonate the construction with two ordinary things: command-line tools, and `async` event handlers.

The impersonation has three parts, and each one has appeared above.

A container (P, R) becomes a tagged union together with a lookup indexed by its tags. That works, and works only, because the tag set is *finite*: over a finite index a dependent type is just a table, and a table is something TypeScript can write down.

The inhabitant $(p : P) \to R\,p$ becomes the handler record, whose totality the compiler enforces because it is a mapped type over that same union.

And a morphism becomes an `async` function — which is where the two directions meet:

```
const reply = await lower.run(u(prompt));   // the prompt, going down
return f(prompt, reply);                    // the response, coming back
```

The response flows backwards

That is the part worth dwelling on. Control flows *down*: a handler is woken by a prompt, computes a lower-level prompt with u, and fires it — which in this implementation means spawning a process and opening a socket, so the descent is a real one, made of real programs. Then it stops. At the `await` the handler suspends, hands the runtime back, and the runtime is free to wake other handlers while it waits.

The response then flows *backwards* along the path the prompt took. Each reply resumes the handler that fired the prompt, at the very line it stopped on, and that handler applies f to turn the reply into its own. So u runs on the way down and f on the way back up, in one function body, read top to bottom, with the suspension between them invisible in the text.

The arrows in the trace are exactly this: ↓ is a prompt descending as a process, ↑ is a response climbing back as a resumed function. It is also why a server here can be a client without ceasing to be a server — the Plan sits mid-answer with four prompts outstanding beneath it, still listening, because suspending is not the same as being busy.

And where the fake fails, it fails visibly. Sequential composition cannot be sent over the wire, because its shape contains a function and functions do not serialise. A validated reply is guaranteed to have the right shape and is guaranteed nothing about its truth. Both limits are consequences of the same substrate that made the impersonation possible in the first place, which is the honest summary of the whole exercise: the algebra is real, and everything holding it up is Unix.